

APPSYNC

AWS AppSync – DVA-C02 Complete Master Lesson

1. Introduction to AWS AppSync

What is AWS AppSync?

AWS AppSync is a fully managed AWS service that makes it easier to build **GraphQL APIs** for applications.

It allows applications to:

- Query data from multiple sources using a single API endpoint
- Perform real-time data updates
- Synchronize application data across devices
- Implement offline capabilities
- Control access using authentication and authorization mechanisms

Instead of creating multiple REST endpoints such as:

GET /users

GET /orders

GET /products

POST /comments

AppSync provides one GraphQL endpoint:

<https://api-id.appsync-api.region.amazonaws.com/graphql>

The client decides exactly what data it needs.

Example:

```
query {  
  customer(id:"123") {  
    name  
    orders {  
      orderId
```

```
    amount
  }
}
}
```

The API returns:

```
{
  "customer": {
    "name": "John",
    "orders": [
      {
        "orderId": "ORD001",
        "amount": 250
      }
    ]
  }
}
```

Why Was AWS AppSync Created?

Before GraphQL services, developers commonly built REST APIs.

A REST architecture often creates problems:

Problem 1: Over-fetching Data

Example:

Mobile application needs:

User name

Profile picture

REST endpoint:

GET /users/123

Response:

```
{  
  "id":123,  
  "name":"John",  
  "email":"john@example.com",  
  "address":"...",  
  "phone":"...",  
  "preferences":"..."  
}
```

The mobile application receives unnecessary data.

Problems:

- More bandwidth usage
- Slower mobile applications
- Higher costs

Problem 2: Under-fetching Data

Application needs:

Customer

Orders

Payments

Reviews

REST requires multiple calls:

GET /customers/123

GET /orders/customer/123

GET /payments/customer/123

GET /reviews/customer/123

This increases:

- Network latency
- Application complexity

Problem 3: Real-Time Applications Are Difficult

Examples:

- Chat applications
- Collaboration tools
- Stock dashboards
- Multiplayer games

Traditional REST APIs require:

- Polling
- WebSockets
- Custom connection management

AppSync provides managed real-time capabilities.

What Problem Does AppSync Solve?

AppSync provides:

Problem	AppSync Solution
Multiple API endpoints	Single GraphQL endpoint
Data over-fetching	Client selects fields

Problem	AppSync Solution
Real-time updates	GraphQL subscriptions
Multiple data sources	Resolvers
Offline applications	Client synchronization
Authentication	Built-in auth integration
API scaling	Fully managed service

Real-World Analogy

Imagine a restaurant.

REST API Model

You have separate waiters:

Waiter 1 → Food

Waiter 2 → Drinks

Waiter 3 → Dessert

You must talk to each waiter separately.

AppSync GraphQL Model

You have a personal assistant.

You say:

"I want today's lunch menu, calories, and available desserts."

The assistant contacts:

- Kitchen
- Inventory system
- Nutrition database

and returns exactly what you requested.

The assistant is AppSync.

Why Companies Use AWS AppSync

Companies use AppSync because it provides:

1. Faster Application Development

Developers do not need to build:

- API routing
 - WebSocket servers
 - Subscription infrastructure
 - Authentication flows
-

2. Mobile-Friendly APIs

Mobile applications benefit from:

- Reduced payload size
- Offline synchronization
- Real-time updates

Common users:

- Banking apps
 - E-commerce apps
 - Healthcare apps
 - Social applications
-

3. Microservices Aggregation

A single GraphQL API can combine:

AppSync

|

|

| | |

DynamoDB Lambda Aurora

| | |

Users Business Reports

The frontend does not need to know backend complexity.

Real-World Business Examples

Example 1: E-Commerce Application

A shopping application needs:

Customer:

Name

Orders

Recommendations

Reviews

Inventory

Data exists in:

DynamoDB

Aurora

OpenSearch

Lambda

AppSync combines everything.

Example 2: Healthcare Portal

A patient dashboard needs:

Patient profile

Appointments

Medical records

Lab results

Doctor messages

AppSync provides one secure API.

Example 3: Real-Time Collaboration

Applications like:

- Document editing
- Chat
- Whiteboards

need instant updates.

Architecture:

User A

|

|

AppSync Subscription

|

|

User B

Changes appear immediately.

AWS AppSync in DVA-C02 Exam Context

For the AWS Certified Developer Associate exam, you should understand:

Core Exam Areas

1. GraphQL Concepts

Know:

- Queries
 - Mutations
 - Subscriptions
 - Schema
 - Resolvers
-

2. Data Sources

AppSync can connect to:

- DynamoDB
 - Lambda
 - Aurora Serverless
 - HTTP endpoints
 - OpenSearch
 - EventBridge
-

3. Authentication

Important exam topics:

- API Key
 - IAM
 - Amazon Cognito User Pools
 - OpenID Connect
 - Lambda Authorization
-

4. Resolver Types

Know:

- Unit resolvers
- Pipeline resolvers
- VTL templates
- JavaScript resolvers

5. Real-Time Features

Understand:

- GraphQL subscriptions
- WebSocket connections
- Pub/Sub model

AppSync Architecture Overview

Basic architecture:

Client Application

Web / Mobile / IoT Application

|

|

GraphQL Request

|

|

AWS AppSync API



DynamoDB Lambda Aurora



Data Sources

Core AppSync Components

1. GraphQL Schema

The schema defines:

- Available operations
- Data types
- Relationships

Example:

```
type User {  
  id: ID!  
  name: String!  
  email: String  
}
```

```
type Query {
```

```
getUser(id: ID!): User
}
```

Meaning:

The API supports:

```
getUser()
```

and returns:

User

2. Resolver

A resolver connects GraphQL fields to backend data.

Example:

GraphQL:

```
query {
  getUser(id:"123")
}
```

Resolver:

```
getUser
```

```
|
```

```
|
```

DynamoDB GetItem

```
|
```

```
|
```

Return User

3. Data Source

The backend service containing data.

Supported sources:

Data Source	Example Usage
-------------	---------------

DynamoDB	Application data
----------	------------------

Lambda	Custom business logic
--------	-----------------------

Aurora	Relational data
--------	-----------------

HTTP API	External APIs
----------	---------------

OpenSearch	Search
------------	--------

None	Local operations
------	------------------

4. Operations

GraphQL has three main operations.

Query

Used for reading data.

Example:

```
query {  
  getProduct(id:"10"){  
    name  
    price  
  }  
}
```

Equivalent:

GET request

Mutation

Used for changing data.

Example:

```
mutation {  
  createProduct(  
    name:"Laptop"  
    price:1000  
  )  
}
```

Equivalent:

POST

PUT

DELETE

Subscription

Used for real-time updates.

Example:

```
subscription {  
  onCreateMessage {  
    message  
  }  
}
```

When a message is created:

Server pushes update

5. AppSync API Types

GraphQL API

Traditional AppSync API.

Features:

- Queries
- Mutations
- Subscriptions
- Resolvers

Events API

AppSync also supports event-driven APIs.

Used for:

- Pub/Sub
- Real-time events

Architecture:

Publisher

|

|

AppSync Events

|

|

Subscribers

Advantages of AWS AppSync

1. Fully Managed

AWS manages:

- Infrastructure
- Scaling

- Availability
 - WebSocket connections
-

2. Real-Time Support

No need to manage:

- WebSocket servers
 - Connection states
 - Broadcasting
-

3. Multiple Backend Sources

One API can combine:

DynamoDB

+

Lambda

+

Aurora

4. Fine-Grained Authorization

Supports:

- User-based access
 - Role-based access
 - Field-level authorization
-

Limitations of AWS AppSync

1. GraphQL Complexity

Large schemas can become difficult to maintain.

2. Resolver Complexity

Poor resolver design can cause:

- Slow APIs
- Excessive database calls

3. Learning Curve

Developers must understand:

- GraphQL
- Schema design
- Resolver mapping

4. Not Always Best for Simple APIs

For simple CRUD APIs:

Amazon API Gateway + Lambda may be simpler.

When to Use AWS AppSync

Use AppSync when you need:

- ✓ GraphQL APIs
- ✓ Mobile applications
- ✓ Real-time updates
- ✓ Multiple backend sources
- ✓ Offline synchronization
- ✓ Complex data relationships

When NOT to Use AppSync

Avoid AppSync when:

- ✗ Simple REST API is enough
 - ✗ Application does not need GraphQL
 - ✗ Team has no GraphQL experience
 - ✗ Very simple backend integration
-

Scalability

AppSync automatically scales:

- API requests
- WebSocket connections
- Resolver execution

You do not manage:

- Servers
 - Load balancers
 - Connection pools
-

Availability

AWS AppSync provides:

- Multi-AZ infrastructure
- Managed service availability
- Regional deployment

For global applications:

Use:

CloudFront

+

Multiple AppSync APIs

+

Route 53

Performance Considerations

1. Avoid N+1 Queries

Bad:

Get users

For each user:

Get orders

Creates:

1 + N database calls

Better:

- Batch requests
- Pipeline resolvers
- Optimized DynamoDB access patterns

2. Use Caching

AppSync supports:

- API caching
- Resolver caching

Benefits:

- Reduced latency
- Lower backend load

3. Optimize Schema Design

Avoid:

- Huge nested queries

- Unnecessary fields

4. Use Pagination

For large datasets:

Bad:

`getAllUsers`

Better:

`getUsers(limit:20,nextToken:"abc")`

DVA-C02 Exam Memory Points

Remember:

Concept	Exam Keyword
Read data	Query
Modify data	Mutation
Real-time updates	Subscription
Backend connection	Resolver
Data storage	Data Source
Mobile sync	AppSync
GraphQL API security	Authorization modes
Multiple backends	AppSync resolvers

3. AWS AppSync Console Walkthrough

In this section, we will create a complete AWS AppSync GraphQL API using the AWS Management Console.

We will create:

- GraphQL API

- Schema
- DynamoDB data source
- Resolver
- Authentication configuration
- Test queries

Architecture we will build:

Client

|

|

GraphQL Request

|

|

AWS AppSync

|

|

Resolver

|

|

DynamoDB Table

|
|

User Records

Prerequisites

Before creating an AppSync API, you need:

AWS Account

Required permissions:

```
{  
  "Effect": "Allow",  
  "Action": [  
    "appsync:*",  
    "dynamodb:*",  
    "iam:PassRole"  
  ],  
  "Resource": "*"   
}
```

For production environments, avoid using full permissions.

Use least privilege.

Step 1: Open AWS AppSync Console

1. Sign in to AWS Management Console.
2. Search:

AppSync

3. Select:

AWS AppSync

You will see:

APIs

Data Sources

Functions

Resolvers

Schema

Settings

Step 2: Create a GraphQL API

Click:

Create API

You will see API options.

Choose:

GraphQL APIs

API Creation Options

You have options:

Option 1: Design from scratch

Choose:

Create with API Designer

Used when:

- You already know your schema
 - Production APIs
-

Option 2: Import schema

Upload:

schema.graphql

Useful when:

- Existing GraphQL application
 - CI/CD deployment
-

Option 3: Create with DynamoDB

AWS generates:

- Schema
- Tables
- Resolvers

Good for learning.

For this walkthrough:

Select:

Design from scratch

Step 3: Configure API Details

Enter:

API Name

Example:

CustomerAPI

Authentication Mode

Options:

Authentication Option 1: API Key

Example:

API_KEY

How it works:

Client sends:

x-api-key:

xxxxxxx

Use cases:

- Development
- Public APIs
- Testing

Do NOT use for sensitive applications.

Authentication Option 2: IAM

Uses AWS Signature Version 4.

Example clients:

- Lambda
- AWS services
- Backend applications

Request contains:

Authorization

X-Amz-Date

X-Amz-Signature

Best for:

- Server-to-server communication
-

Authentication Option 3: Amazon Cognito User Pools

Used for:

- Mobile applications
- Web applications
- User authentication

Flow:

User Login

|
|

Cognito User Pool

|
|

JWT Token

|
|

AppSync

Authentication Option 4: OpenID Connect

Used with:

- Auth0

- Okta
- Enterprise identity providers

Authentication Option 5: Lambda Authorization

Custom authorization logic.

Example:

Client

|

AppSync

|

Lambda Authorizer

|

Allow/Deny

Useful when:

- Custom token validation required

Recommended Learning Configuration

For testing:

Select:

API Key

Why?

Because it requires no external identity provider.

Step 4: Create API

Click:

Create API

AWS creates:

- GraphQL endpoint
 - API key
 - Empty schema
 - IAM service role
-

Step 5: Define GraphQL Schema

Navigate:

Schema

Replace default schema:

```
type User {
```

```
  id: ID!
```

```
  name: String!
```

```
  email: String
```

```
}
```

```
type Query {
```

```
  getUser(id: ID!): User
```

```
}
```

```
type Mutation {  
  createUser(  
    id: ID!,  
    name: String!,  
    email: String!  
  ): User  
}
```

```
schema {  
  query: Query  
  mutation: Mutation  
}
```

Understanding This Schema

Type User

Defines object structure:

```
type User
```

Example response:

```
{  
  "id": "100",  
  "name": "Alice",  
  "email": "alice@example.com"  
}
```

Query

Read operation:

```
getUser(id:ID!)
```

Example:

```
query {  
  getUser(id:"100"){  
    name  
    email  
  }  
}
```

Mutation

Write operation:

```
createUser()
```

Example:

```
mutation {  
  createUser(  
    id:"100",  
    name:"Alice",  
    email:"alice@example.com"  
  )  
}
```

Step 6: Create DynamoDB Data Source

Go to:

Data Sources

Click:

Create Data Source

Configure:

Data Source Name

Example:

UserTable

Data Source Type

Select:

Amazon DynamoDB table

Create Table

Choose:

Create new table

Table name:

Users

Partition key:

id

Type:

String

Example data:

id	name	email
----	------	-------

100	Alice	alice@test.com
-----	-------	----------------

101	Bob	bob@test.com
-----	-----	--------------

IAM Role

Select:

Create new role

AWS creates permissions:

```
{  
  "Effect": "Allow",  
  "Action": [  
    "dynamodb:GetItem",  
    "dynamodb:PutItem",  
    "dynamodb:Query"  
  ],  
  "Resource": "table ARN"  
}
```

Step 7: Create Resolver

A resolver connects:

GraphQL Field

|

Resolver

|

DynamoDB

Go to:

Schema

Find:

Query

|

getUser

Click:

Attach Resolver

Select:

Data source:

Users

Resolver Type:

Unit Resolver

Resolver Request Mapping Template

Example:

{

"version": "2018-05-29",

"operation": "GetItem",

```
"key":{  
  "id":{  
    "S": "$ctx.args.id"  
  }  
}  
}
```

Meaning:

Take GraphQL argument:

```
id:"100"
```

Convert into:

```
{  
  "id":{  
    "S": "100"  
  }  
}
```

Resolver Response Mapping

```
$util.toJson($ctx.result)
```

Returns DynamoDB response.

Step 8: Test GraphQL Query

Go to:

Queries

Enter:

```
query GetUser {
```

```
getUser(id:"100") {
```

```
  id
```

```
  name
```

```
  email
```

```
}
```

```
}
```

Click:

Run

Expected:

```
{
```

```
  "data":{
```

```
    "getUser":{
```

```
      "id":"100",
```

```
      "name":"Alice",
```

```
      "email":"alice@test.com"
```

```
    }
```

```
  }
```

```
}
```

Common Console Mistakes

Mistake 1: Schema deployed but resolver missing

Error:

Cannot return null for non-nullable field

Cause:

GraphQL field has no resolver.

Solution:

Attach resolver.

Mistake 2: DynamoDB permissions missing

Error:

AccessDeniedException

Cause:

AppSync IAM role cannot access DynamoDB.

Solution:

Add:

dynamodb:GetItem

dynamodb:PutItem

Mistake 3: Wrong partition key

Example:

DynamoDB:

userId

Resolver:

id

Result:

Item not found

Mistake 4: API key expired

API keys have expiration.

Production:

Use:

- Cognito
- IAM
- OIDC

Production Console Best Practices

Enable Logging

Navigate:

Settings

→ Logging

→ Enable CloudWatch Logs

Recommended:

Field Resolver Logs:

ALL

During development.

Production:

ERROR

to reduce cost.

Enable X-Ray

Enable:

AWS X-Ray tracing

Useful for:

- Latency analysis
 - Resolver debugging
-

Enable API Cache

For read-heavy workloads:

Settings

→ API Cache

→ Enable

Example:

Product catalog.

4. AWS CLI with AppSync

AWS CLI is useful for:

- Automation
 - CI/CD
 - Infrastructure management
-

Install AWS CLI

Verify:

`aws --version`

Example:

`aws-cli/2.x`

Configure Credentials

Command:

```
aws configure
```

Enter:

AWS Access Key ID

AWS Secret Access Key

Default region

Output format

Example:

Region:

us-east-1

Create AppSync API Using CLI

Command:

```
aws appsync create-graphql-api \
```

```
--name CustomerAPI \
```

```
--authentication-type API_KEY
```

Explanation

create-graphql-api

Creates GraphQL API.

--name

API name.

Example:

CustomerAPI

--authentication-type

Authentication mode.

Values:

API_KEY

AWS_IAM

AMAZON_COGNITO_USER_POOLS

OPENID_CONNECT

AWS_LAMBDA

Expected output:

```
{
  "graphqlApi":{
    "apiId":"abc123",
    "name":"CustomerAPI",
    "authenticationType":"API_KEY"
  }
}
```

List APIs

Command:

```
aws appsync list-graphql-apis
```

Output:

```
{
  "graphqlApis":[
    {
      "name":"CustomerAPI",
```

```
"apiId":"abc123"
}
]
}
```

Get API Details

```
aws appsync get-graphql-api \
--api-id abc123
```

Returns:

- Endpoint
 - ARN
 - Authentication
-

Create API Key

Command:

```
aws appsync create-api-key \
--api-id abc123
```

Output:

```
{
  "apiKey":{
    "id":"key123",
    "expires":123456789
  }
}
```

Upload Schema

Create file:

schema.graphql

Command:

aws appsync start-schema-creation \

--api-id abc123 \

--definition fileb://schema.graphql

Common CLI Errors

Error

AccessDeniedException

Cause:

IAM permissions missing.

Need:

appsync:CreateGraphQLApi

Error

BadRequestException

Cause:

Invalid schema.

Check:

- Missing type
- Invalid syntax
- Missing Query root

CLI Exam Notes

DVA-C02 often tests:

- IAM permissions
- Authentication type
- Resolver configuration
- Automation using SDK/CLI

5. AWS AppSync SDK Examples (Python boto3)

In this section, we will learn how developers interact with AWS AppSync programmatically.

Important DVA-C02 point:

boto3 does not execute GraphQL queries directly. It manages AppSync resources (APIs, schemas, API keys, resolvers). Applications execute GraphQL operations by calling the AppSync GraphQL endpoint.

5.1 Install boto3

Install:

```
pip install boto3
```

Verify:

```
import boto3
```

```
print(boto3.__version__)
```

5.2 Authentication Setup

boto3 uses AWS credentials.

Credential sources:

Priority order:

1. Environment variables
2. IAM Role attached to EC2/ECS/Lambda
3. AWS credentials file
4. IAM Identity Center credentials

Example:

```
export AWS_ACCESS_KEY_ID="ACCESS_KEY"
```

```
export AWS_SECRET_ACCESS_KEY="SECRET_KEY"
```

```
export AWS_DEFAULT_REGION="us-east-1"
```

Recommended Production Authentication

Avoid:

```
AWS_ACCESS_KEY_ID="hardcoded"
```

Never store credentials inside:

- Source code
- Git repositories
- Docker images

Preferred:

Environment	Recommended Method
Lambda	Execution Role
ECS	Task Role
EC2	Instance Role
Developer Machine	IAM Identity Center

5.3 Create AppSync Client

Python:

```
import boto3
```

```
appsync_client = boto3.client(  
    "appsync",  
    region_name="us-east-1"  
)
```

```
print(appsync_client)
```

Explanation:

boto3.client()

Creates a connection to AWS service APIs.

"appsync"

Selects AppSync management API.

region_name

Specifies AWS region.

Example:

us-east-1

5.4 Create GraphQL API Using boto3

Code:

```
import boto3
```

```
client = boto3.client("appsync")
```

```
response = client.create_graphql_api(  
    name="CustomerAPI",  
    authenticationType="API_KEY"  
)
```

```
print(response)
```

Explanation:

create_graphql_api()

Creates a new AppSync API.

name

API display name.

Example:

```
name="CustomerAPI"
```

authenticationType

Options:

API_KEY

AWS_IAM

AMAZON_COGNITO_USER_POOLS

OPENID_CONNECT

AWS_LAMBDA

Example output:

```
{  
  "graphqlApi": {  
    "apiId": "abc123",  
    "name": "CustomerAPI",  
    "authenticationType": "API_KEY"  
  }  
}
```

5.5 List GraphQL APIs

Code:

```
response = client.list_graphql_apis()
```

```
for api in response["graphqlApis"]:  
    print(api["name"])
```

Output:

CustomerAPI

ProductAPI

OrderAPI

5.6 Get API Details

Code:

```
response = client.get_graphql_api(  
    apiId="abc123"  
)
```

```
print(  
    response["graphqlApi"]["uris"]  
)
```

Example output:

```
{  
  "GRAPHQL":  
    "https://abc123.appsync-api.us-east-1.amazonaws.com/graphql"  
}
```

5.7 Create API Key

Example:

```
response = client.create_api_key(  
    apiId="abc123",  
    description="Development Key",  
    expires=1735689600  
)
```

```
print(response)
```

Parameters:

apild

Target AppSync API.

description

Human-readable identifier.

Example:

Development Key

expires

Unix timestamp.

Example:

1735689600

5.8 Error Handling

Production code should handle AWS exceptions.

Example:

```
import boto3
```

```
from botocore.exceptions import ClientError
```

```
client = boto3.client("appsync")
```

try:

```
    response = client.get_graphql_api(
```

```
        apId="invalid"  
    )
```

except ClientError as error:

```
    print(  
        error.response["Error"]["Code"]  
    )
```

```
    print(  
        error.response["Error"]["Message"]  
    )
```

Possible errors:

Error	Meaning
AccessDeniedException	IAM problem
NotFoundException	API does not exist
BadRequestException	Invalid request
LimitExceededException	Service quota exceeded

5.9 Executing GraphQL Queries with Python

For applications, you normally call the AppSync endpoint.

Install:

```
pip install requests
```

Example:

```
import requests
```

```
url = "https://api-id.appsync-api.us-east-1.amazonaws.com/graphql"
```

```
query = ""
```

```
query {
```

```
  listUsers {
```

```
    id
```

```
    name
```

```
  }
```

```
}
```

```
""
```

```
response = requests.post(
```

```
    url,
```

```
    json={
```

```
    "query": query
},
headers={
    "x-api-key": "YOUR_API_KEY"
}
)
```

```
print(response.json())
```

Flow:

Python Application

|

|

HTTPS Request

|

|

AWS AppSync

|

|

Resolver

|
|

Backend Data Source

Production SDK Best Practices

1. Use IAM Roles

Bad:

```
secret="abc123"
```

Good:

```
boto3.client("appsync")
```

with IAM role.

2. Enable Retries

boto3 automatically retries some AWS API failures.

Custom configuration:

```
from botocore.config import Config
```

```
config = Config(
```

```
    retries={
```

```
        "max_attempts":5,
```

```
        "mode":"adaptive"
```

```
    }
```

```
)
```

```
client=boto3.client(  
    "appsync",  
    config=config  
)
```

3. Use Environment Variables

Example:

```
import os
```

```
region=os.environ["AWS_REGION"]
```

6. Infrastructure as Code

Infrastructure as Code allows teams to create AppSync APIs consistently.

We cover:

1. CloudFormation
 2. AWS SAM
 3. AWS CDK
-

6.1 AWS CloudFormation

CloudFormation resources:

```
AWS::AppSync::GraphQLApi
```

```
AWS::AppSync::GraphQLSchema
```

AWS::AppSync::DataSource

AWS::AppSync::Resolver

CloudFormation Example

AWSTemplateFormatVersion: "2010-09-09"

Resources:

CustomerAPI:

Type:

AWS::AppSync::GraphQLApi

Properties:

Name:

CustomerAPI

AuthenticationType:

API_KEY

Explanation

AWS::AppSync::GraphQLApi

Creates AppSync API.

Name

API name.

AuthenticationType

Defines authentication.

Possible values:

API_KEY

AWS_IAM

AMAZON_COGNITO_USER_POOLS

GraphQL Schema Resource

Example:

Schema:

Type:

AWS::AppSync::GraphQLSchema

Properties:

ApiId:

!GetAtt CustomerAPI.ApiId

Definition: |

```
type User {
```

```
  id: ID!
```

```
  name: String!
```

```
}
```

```
type Query {
```

```
  getUser(id:ID!):User
```

```
}
```

```
schema {
```

```
  query:Query
```

```
}
```

Resolver Example

GetUserResolver:

Type:

AWS::AppSync::Resolver

Properties:

ApiId:

!GetAtt CustomerAPI.ApiId

TypeName:

Query

FieldName:

getUser

DataSourceName:

UserTable

CloudFormation Deployment

Package:

aws cloudformation package

Deploy:

```
aws cloudformation deploy \
```

```
--template-file template.yaml \
```

```
--stack-name CustomerAPIStack
```

CloudFormation Best Practices

Use:

- Parameters
- Outputs
- IAM least privilege
- Stack policies
- Change sets

6.2 AWS SAM with AppSync

AWS SAM simplifies serverless deployments.

Note:

SAM support for AppSync is commonly done through:

- `AWS::Serverless::GraphQLApi`
- CloudFormation resources

Example:

Resources:

CustomerAPI:

Type:

AWS::Serverless::GraphQLApi

Properties:

Name:

CustomerAPI

Auth:

DefaultAuthorization:

AuthorizationType:

API_KEY

Benefits:

- Easier deployment
 - Local testing
 - CI/CD integration
-

SAM Deployment

Build:

sam build

Deploy:

sam deploy --guided

6.3 AWS CDK TypeScript

CDK allows developers to define infrastructure using programming languages.

Install:

```
npm install -g aws-cdk
```

Create stack:

```
cdk init app --language typescript
```

Install AppSync package:

```
npm install aws-cdk-lib
```

Example:

```
import { Stack } from 'aws-cdk-lib';  
  
import { GraphQLApi,  
SchemaFile,  
AuthorizationType } from 'aws-cdk-lib/aws-appsync';
```

```
export class CustomerApiStack extends Stack {
```

```
  constructor(scope:any,id:string){
```

```
    super(scope,id);
```

```
    const api =
```

```
new GraphQLApi(  
  this,  
  "CustomerAPI",  
  {  
  
    name:"CustomerAPI",  
  
    schema:  
      SchemaFile.fromAsset(  
        "schema.graphql"  
      ),  
  
    authorizationConfig:{  
      defaultAuthorization:{  
        authorizationType:  
          AuthorizationType.API_KEY  
      }  
    }  
  
  });  
  
}  
  
}
```

Explanation:

GraphQLApi

Creates AppSync API.

SchemaFile

Loads GraphQL schema.

AuthorizationType.API_KEY

Uses API key authentication.

CDK Deployment

Bootstrap:

cdk bootstrap

Deploy:

cdk deploy

Infrastructure as Code Exam Points

DVA-C02 frequently tests:

Question Pattern	Answer
Repeat environments	CloudFormation/CDK
Avoid manual console changes IaC	
Serverless deployment	SAM
Programmatic infrastructure	CDK

7. IAM & Security for AWS AppSync

Security is one of the most important DVA-C02 exam areas.

AWS AppSync security has two major layers:

1. **Authentication** — Who are you?
 2. **Authorization** — What are you allowed to do?
-

7.1 Authentication vs Authorization

Authentication

Answers:

"Who is making the request?"

Examples:

- User login
 - API key
 - IAM identity
 - JWT token
-

Authorization

Answers:

"What can this user access?"

Example:

User:

Alice

Permissions:

Can read:

- Own profile

Cannot read:

- Other users' profiles

AppSync Authorization Modes

AppSync supports:

Authorization Mode	Purpose	Common Use Case
API Key	Simple access control	Development, public APIs
IAM	AWS identity based access	Backend services
Cognito User Pool	User authentication	Mobile/web applications
OpenID Connect	External identity providers	Enterprise applications
Lambda Authorizer	Custom authorization logic	Custom security rules

7.2 API Key Authentication

Flow:

Client

|

|

x-api-key header

|

|

AWS AppSync

|

|

GraphQL Resolver

Example:

HTTP header:

x-api-key:

da2-example-key

Advantages

- Simple
- Easy testing
- No user management

Disadvantages

- Shared secret
- No user identity
- Not suitable for sensitive applications

Exam Trap

Question:

A developer wants to expose a production banking API. Which authentication should be used?

Incorrect:

API_KEY

Better:

Cognito User Pool

IAM

depending on architecture.

7.3 IAM Authentication

IAM authentication uses AWS Signature Version 4.

Flow:

Lambda Function

|

|

AWS SDK signs request

|

|

AppSync API

|

|

IAM Authorization

Example IAM policy:

{

"Version": "2012-10-17",

```
"Statement":[

{

  "Effect":"Allow",


  "Action":[

    "appsync:GraphQL"

  ],


  "Resource":

    "arn:aws:appsync:us-east-1:123456789012:apis/api-id/*"

}

]

}
```

Explanation:

appsync:GraphQL

Allows executing GraphQL operations.

Resource

Limits access to:

- Specific API
 - Specific fields
-

7.4 Cognito User Pool Authentication

Common architecture:

User

|

|

Login

|

|

Amazon Cognito

|

|

JWT Token

|

|

AWS AppSync

|

|

Resolver

JWT contains:

```
{  
  "sub":"user-id",  
  "email":"user@example.com",  
  "groups":[  
    "Admin"  
  ]  
}
```

Resolvers can use:

`$ctx.identity`

to access user information.

Example Authorization Logic

Only admins can delete users:

```
if(ctx.identity.groups.includes("Admin")){  
  allow()  
}  
else{  
  deny()  
}
```

7.5 Lambda Authorizer

Used when authorization rules are complex.

Example:

Company has:

Employee

Manager

Contractor

Partner

Rules cannot easily be represented by Cognito groups.

Architecture:

Client

|

|

Token

|

|

AppSync

|

|

Lambda Authorizer

|

|

Allow / Deny

Lambda returns:

```
{  
  "isAuthorized":true,  
  
  "resolverContext":{  
    "role":"manager"  
  }  
}
```

7.6 Resource Policies

AppSync supports IAM-based restrictions.

Example:

Allow only specific AWS account:

```
{  
  "Effect":"Allow",  
  
  "Principal":{  
    "AWS":  
      "arn:aws:iam::111111111111:root"  
  },  
  
  "Action":  
    "appsync:GraphQL",
```

"Resource":

"*"

}

7.7 Least Privilege IAM

Avoid:

{

"Effect": "Allow",

"Action": "*",

"Resource": "*"

}

Better:

{

"Effect": "Allow",

"Action": [

"dynamodb:GetItem",

"dynamodb:Query"

],

"Resource":

"arn:aws:dynamodb:us-east-1:123456789012:table/Users"

}

7.8 Encryption

Encryption in Transit

AppSync uses:

HTTPS/TLS

All GraphQL requests are encrypted.

Encryption at Rest

AWS encrypts:

- API configuration
 - Logs
 - Cached data
-

7.9 AWS KMS Integration

KMS is used for encryption key management.

Architecture:

AppSync

|

|

KMS Key

|

|

Encrypted Data

Benefits:

- Customer managed keys
- Key rotation
- Audit control

7.10 Secrets Manager

Use for:

- Database passwords
- API credentials
- External service secrets

Example:

Bad:

password="Admin123"

Good:

Application

|

|

Secrets Manager

|

|

Database Password

7.11 Systems Manager Parameter Store

Alternative for:

- Configuration values
- Environment variables
- Non-sensitive settings

Example:

/production/appsync/database-url

Security Best Practices

1. Use Cognito for End User Applications

Reason:

Provides:

- User identity
 - JWT tokens
 - Groups
-

2. Use IAM for AWS Service Communication

Example:

Lambda → AppSync

3. Enable CloudTrail

Track:

- API changes
 - Configuration changes
-

4. Enable Logging Carefully

Development:

ALL

Production:

ERROR

5. Validate GraphQL Inputs

Prevent:

- Injection attacks
- Unexpected data

6. Limit Query Complexity

Prevent:

- Expensive nested queries
- Resource exhaustion

Common Security Mistakes

Mistake	Problem	Solution
Using API keys in production	Shared secret	Use Cognito/IAM
Hardcoded credentials	Credential leak	IAM roles
Over-permissioned IAM roles	Security risk	Least privilege
No CloudTrail	Poor auditing	Enable logging
No encryption	Compliance issue	KMS
Unlimited queries	DoS risk	Query limits

8. AWS AppSync Pricing

AppSync pricing is based mainly on:

1. GraphQL operation requests
2. Real-time connections
3. Data transfer
4. Caching
5. Optional features

Pricing changes over time, so always verify current pricing before production decisions.

8.1 GraphQL Requests

Charges apply for:

- Queries
- Mutations

Example:

Application:

10 million queries/month

Cost depends on:

- Region
 - Request volume
-

8.2 Real-Time Connection Pricing

Subscriptions maintain WebSocket connections.

Cost factors:

- Connection minutes
- Messages delivered

Example:

Chat application:

100,000 users connected

will create significant connection usage.

8.3 Data Transfer Costs

Possible charges:

- Internet data transfer
- Cross-region transfer

Optimize by:

- CloudFront
 - Regional deployment
 - Reducing payload size
-

8.4 API Caching Costs

Caching adds:

- Cache instance cost

Benefits:

- Lower DynamoDB usage
 - Faster responses
-

Cost Optimization Best Practices

1. Enable Caching

For:

- Product catalogs
 - Read-heavy APIs
-

2. Reduce Payload Size

GraphQL advantage:

Client requests only needed fields.

3. Optimize Resolvers

Avoid:

One request

=

100 database calls

4. Monitor Usage

Use:

- CloudWatch
 - Cost Explorer
 - Billing alarms
-

9. Monitoring and Troubleshooting

Production AppSync applications require visibility.

Important AWS services:

- CloudWatch
 - CloudTrail
 - X-Ray
-

9.1 CloudWatch Metrics

AppSync publishes metrics.

Important metrics:

Metric	Meaning
4XXError	Client errors
5XXError	Server errors
Latency	API response time
Requests	Request count
Connected clients	Subscription usage

9.2 CloudWatch Logs

Enable:

AppSync Settings

→ Logging

→ Enable CloudWatch Logs

Log levels:

NONE

No logs.

ERROR

Production recommendation.

ALL

Debugging.

Example error:

DynamoDB AccessDeniedException

Log shows:

Resolver failed

Missing dynamodb:GetItem permission

9.3 AWS CloudTrail

CloudTrail records:

- API creation
- Schema updates
- Authentication changes

Example event:

CreateGraphQLApi

UpdateGraphQLApi

DeleteGraphQLApi

9.4 AWS X-Ray

Useful for tracing:

Client

|

AppSync

|

Resolver

|

Lambda

|

DynamoDB

Finds:

- Slow resolver
- Backend latency
- Errors

Troubleshooting Scenarios

Problem 1

Error:

Cannot query field "email"

Cause:

Schema does not contain field.

Solution:

Update GraphQL schema.

Problem 2

Error:

AccessDeniedException DynamoDB

Cause:

AppSync service role missing permission.

Solution:

Add:

"dynamodb:GetItem"

Problem 3

Slow API response

Possible causes:

- Slow Lambda
 - Poor DynamoDB design
 - Missing caching
 - N+1 queries
-

Problem 4

Subscription not receiving events

Check:

- Subscription resolver
 - Mutation exists
 - Client connection
 - Authorization
-

DVA-C02 Monitoring Exam Tips

Remember:

Requirement	Service
-------------	---------

API logs	CloudWatch Logs
----------	-----------------

API changes	CloudTrail
-------------	------------

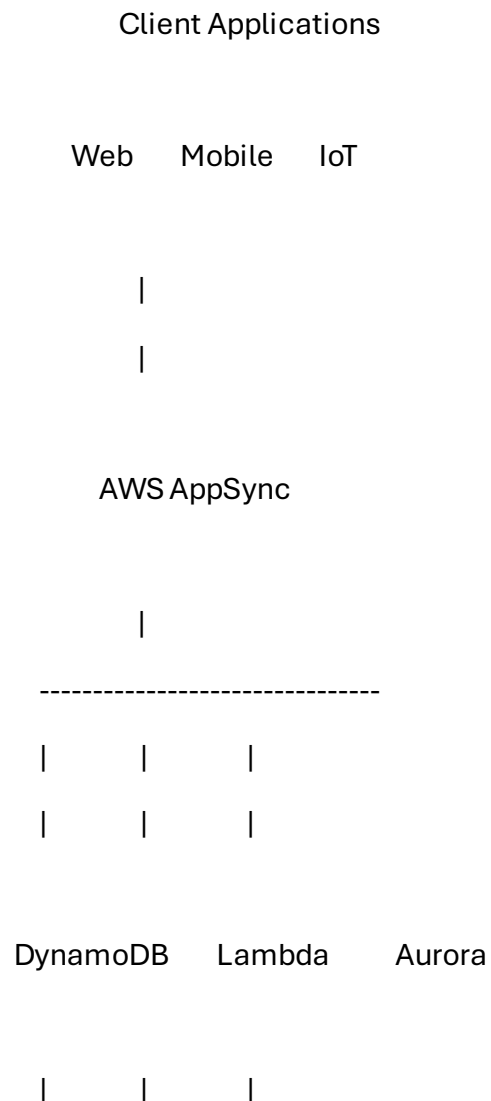
Request tracing	X-Ray
-----------------	-------

Metrics	CloudWatch Metrics
---------	--------------------

10. AWS AppSync Service Integrations

AWS AppSync becomes powerful because it acts as an abstraction layer between clients and multiple backend services.

Architecture:



Application	Business	Relational
Data	Logic	Database

AppSync + Lambda Integration

Why Use Lambda with AppSync?

Lambda is used when you need:

- Custom business logic
- Data transformation
- External API calls
- Complex processing

Example:

User requests:

```
query {  
  calculateShipping(orderId:"123")  
}
```

Flow:

Client

|

|

AppSync

|

|

Lambda Resolver

|

|

Shipping Calculation Logic

How It Works

1. Client sends GraphQL request.
 2. AppSync resolver invokes Lambda.
 3. Lambda processes request.
 4. Lambda returns response.
 5. AppSync sends GraphQL response.
-

IAM Permissions

AppSync needs:

```
{  
  "Effect": "Allow",  
  "Action": [  
    "lambda:InvokeFunction"  
  ],  
  "Resource":  
    "arn:aws:lambda:region:account:function:function-name"  
}
```

Lambda execution role requires permissions for backend services.

Example:

Lambda Role

|

|

dynamodb:GetItem

|

|

DynamoDB

Best Practices

Use Lambda when:

- ✓ Business logic is complex
- ✓ Multiple services must be called
- ✓ Data transformation is required

Avoid Lambda when:

- ✗ Simple DynamoDB CRUD is enough
-

AppSync + DynamoDB Integration

DynamoDB is the most common AppSync data source.

Used for:

- User profiles
- Product catalogs

- Orders
- IoT data

Architecture:

Client

|

AppSync

|

Resolver

|

DynamoDB

Operations:

GraphQL DynamoDB Operation

Query GetItem, Query, Scan

Mutation PutItem, UpdateItem, DeleteItem

IAM Permission Example

{

```
"Effect": "Allow",
"Action": [
  "dynamodb:GetItem",
  "dynamodb:PutItem",
  "dynamodb:UpdateItem"
],
"Resource":
  "arn:aws:dynamodb:*:*:table/Users"
}
```

Best Practices

Use:

- Query instead of Scan
- Proper partition keys
- Pagination
- Conditional writes

AppSync + DynamoDB Streams

DynamoDB Streams capture item changes.

Architecture:

DynamoDB

|

|

Stream Event

|

|

Lambda

|

|

AppSync Mutation

|

|

Clients Receive Update

Example:

A customer changes profile:

DynamoDB Update

|

DynamoDB Stream

|

Lambda

|

Publish Event

|

AppSync Subscription

Use cases:

- Real-time dashboards
 - Notifications
 - Data synchronization
-

AppSync + API Gateway

Comparison:

AppSync	API Gateway
GraphQL	REST/WebSocket
Client controls response	Server defines response
Real-time subscriptions built-in	Requires WebSocket setup
Multiple data sources	Backend integrations

Use API Gateway when:

- REST API required
- External API consumers

Use AppSync when:

- GraphQL required
- Real-time data needed

AppSync + S3

S3 is commonly used for:

- File storage
- Images
- Videos
- Documents

Typical pattern:

Mobile App

|

AppSync

|

Request Upload URL

|

S3

Example:

User uploads profile picture:

1. AppSync mutation:

```
mutation {  
  
  createUploadUrl(filename:"photo.jpg")  
}
```

2. Lambda creates pre-signed S3 URL.
3. Client uploads directly to S3.

Benefits:

- Reduced AppSync traffic
- Better scalability

AppSync + EventBridge

EventBridge enables event-driven architectures.

Architecture:

Application

|

EventBridge

|

Lambda

|

AppSync

|

Subscribers

Use cases:

- Order events
- Business workflows
- SaaS integrations

AppSync + SNS

SNS provides pub/sub messaging.

Example:

Order Created

|

SNS Topic

|

Lambda

|

AppSync

|

Users

Use SNS when:

- Multiple subscribers
- Notification fan-out

AppSync + SQS

SQS provides reliable queues.

Pattern:

AppSync

|

Lambda

|

SQS

|

Worker

Used for:

- Background processing
- Long-running tasks

Example:

User uploads video.

Do not process immediately.

Instead:

AppSync

|

SQS

|

Video Processor

AppSync + Step Functions

Used for workflows.

Example:

Order processing:

Order Created

|

Step Functions

|

Payment

|

Inventory

|

Shipping

AppSync starts workflow:

```
mutation {  
  placeOrder()  
}
```

AppSync + CloudWatch

Used for:

- Metrics
- Logs
- Alarms

Flow:

AppSync

|

CloudWatch Logs

|

Dashboard

|

Alarm

AppSync + CloudTrail

Tracks:

- API creation
- Configuration changes
- Security changes

Example:

Administrator changes authentication:

UpdateGraphQLApi

|

CloudTrail Event

AppSync + IAM

IAM controls:

- Who can call GraphQL APIs
- Backend service permissions

Example:

Lambda:

Lambda Role

allows

appsync:GraphQL

AppSync + KMS

Used for:

- Encryption control
- Compliance requirements

Example:

Healthcare application:

Patient Data

|

KMS Key

|

Encrypted Storage

AppSync + Secrets Manager

Used for:

- Database credentials
- API tokens

Example:

Lambda Resolver

|

Secrets Manager

|

Database Password

AppSync + Systems Manager Parameter Store

Used for:

- Configuration values
- Environment settings

Example:

/prod/payment/url

/prod/database/name

AppSync + ECS

ECS can host backend applications.

Architecture:

AppSync

|

HTTP Resolver

|

ECS Service

|

Application Container

Use when:

- Existing container applications
- Microservices

AppSync + ECR

ECR stores container images.

Flow:

Developer

|

Docker Image

|

ECR

|

ECS

|

AppSync HTTP Resolver

AppSync + CloudFront

CloudFront improves global performance.

Architecture:

Users Worldwide

|

CloudFront

|

AppSync

Benefits:

- Reduced latency
- Edge routing

AppSync + Aurora

Aurora is relational database integration.

Architecture:

AppSync

|

Resolver

|

Aurora Serverless

|

SQL Data

Used when:

- Relational queries
 - Transactions
 - Existing SQL databases
-

AppSync + RDS

Traditional relational database integration.

Example:

AppSync

|

Lambda

|

RDS

AppSync + Kinesis

Used for streaming data.

Architecture:

IoT Devices

|

Kinesis

|

Lambda

|

AppSync

|

Dashboard

AppSync + SES

Used for email workflows.

Example:

User Registration

|

AppSync Mutation

|

Lambda

|

SES

|

Email

AppSync + CodePipeline

CI/CD:

Developer

|

CodeCommit/GitHub

|

CodePipeline

|

CloudFormation/CDK

|

AppSync Deployment

AppSync + CodeBuild

Build phase:

Source Code

|

CodeBuild

|

Tests

|

Deploy

AppSync + CodeDeploy

Used for deployment automation.

Example:

New Lambda Resolver

|

CodeDeploy

|

Production

AppSync + CloudFormation

Infrastructure automation:

Template

|

CloudFormation

|

AppSync API

AppSync + AWS CDK

Developers define:

`new GraphQLApi()`

Benefits:

- Version control
 - Reusable components
-

AppSync + AWS SAM

Used for:

- Serverless deployments
 - CI/CD pipelines
-

Integration Comparison Table

Integration	Main Purpose	Common Exam Scenario
Lambda	Custom logic	Complex processing
DynamoDB	NoSQL data	CRUD API
DynamoDB Streams	Change events	Real-time updates
S3	Files	Upload/download
EventBridge	Events	Decoupled architecture
SNS	Fan-out notifications	Multiple consumers
SQS	Queues	Async processing
Step Functions	Workflow	Business processes
CloudWatch	Monitoring	Logs/metrics
CloudTrail	Auditing	API changes
IAM	Authentication	AWS services
KMS	Encryption	Compliance
Secrets Manager	Secrets	Credentials
ECS	Containers	Microservices
CloudFront	Global access	Low latency
Aurora	SQL	Relational data
Kinesis	Streaming	Real-time analytics
SES	Email	Notifications

11. AWS AppSync Architecture Examples

Architecture 1: Simple Mobile Application

Loading diagram...

Request Flow

1. Mobile app sends GraphQL query.
 2. AppSync validates authentication.
 3. Resolver executes DynamoDB operation.
 4. Data returned to client.
-

Architecture 2: Enterprise Application

Loading diagram...

Architecture 3: Real-Time Chat Application

Loading diagram...

Architecture 4: Highly Secure Banking Application

Loading diagram...

Architecture Security Flow

1. User authenticates.
 2. Cognito issues JWT.
 3. AppSync validates token.
 4. Resolver invokes backend.
 5. Data encrypted using KMS.
-

12. AWS AppSync Hands-on Labs

These labs are designed for AWS Certified Developer Associate (DVA-C02) preparation.

They move from beginner level to production-style implementation.

Lab 1: Create a Simple AppSync GraphQL API with DynamoDB

Objective

Build a GraphQL API that:

- Creates users
- Retrieves users
- Stores data in DynamoDB

Architecture:

Client

|

AWS AppSync

|

DynamoDB

|

Users Table

Prerequisites

You need:

- AWS account
- IAM permissions
- AWS CLI configured

Required permissions:

```
{  
  "Effect": "Allow",  
  "Action": [  
    "appsync:*",  
    "dynamodb:*",  
    "iam:PassRole"  
  ],  
  "Resource": "*"   
}
```

Step 1: Create DynamoDB Table

Open:

AWS Console

→ DynamoDB

→ Tables

→ Create table

Configure:

Table name:

Users

Partition key:

id

Type:

String

Example items:

id name email

1 Alice alice@test.com

2 Bob bob@test.com

Step 2: Create AppSync API

Navigate:

AWS Console

→ AppSync

→ Create API

→ GraphQL API

→ Design from scratch

API name:

UserAPI

Authentication:

API Key

(For learning only)

Step 3: Create Schema

Schema:

```
type User {
```

```
  id: ID!
```

```
  name: String!
```

email: String

}

type Query {

getUser(id:ID!):User

}

type Mutation {

createUser(

id:ID!,

name:String!,

email:String

):User

}

schema {

query:Query

mutation:Mutation

}

Step 4: Create Data Source

Go:

Data Sources

→ Create Data Source

Type:

DynamoDB

Table:

Users

Allow AppSync to create IAM role.

Step 5: Create Resolver

Attach resolver:

Query

|

getUser

Data source:

Users

Request mapping:

{

"version":"2018-05-29",

```
"operation":"GetItem",
```

```
"key":{
```

```
  "id":{
```

```
    "S": "$ctx.args.id"
```

```
  }
```

```
}
```

```
}
```

Step 6: Test Query

Query:

```
query {
```

```
  getUser(id:"1"){
```

```
    id
```

```
    name
```

```
    email
```

```
  }
```

```
}
```

Expected:

```
{
```

```
"data":{
  "getUser":{
    "id":"1",
    "name":"Alice",
    "email":"alice@test.com"
  }
}
```

Troubleshooting

Error

Cannot return null

Cause:

Resolver missing.

Solution:

Attach resolver.

Error

AccessDeniedException

Cause:

AppSync IAM role lacks DynamoDB permissions.

Solution:

Add:

dynamodb:GetItem

Lab 2: Add Cognito Authentication

Objective

Replace API Key authentication with user authentication.

Architecture

User

|

Cognito

|

JWT Token

|

AppSync

|

DynamoDB

Step 1: Create Cognito User Pool

Navigate:

Cognito

→ User Pools

→ Create User Pool

Configure:

Sign-in:

Email

Required attributes:

Email

Step 2: Configure AppSync Authentication

Go:

AppSync

→ Settings

→ Authorization

→ Add authorization mode

Select:

Amazon Cognito User Pool

Step 3: Test Request

Client sends:

Authorization:

eyJhbGciOi...

AppSync validates:

JWT Token

Expected Result

Authenticated users can access API.

Unauthenticated users receive:

Unauthorized

Lab 3: Real-Time Chat Application

Objective

Build a real-time messaging system.

Features:

- Send messages
 - Receive updates instantly
-

Architecture

User A

|

Mutation

|

AppSync

|

DynamoDB

|

Subscription

|

User B

Schema

type Message {

id:ID!

text:String!

}

type Mutation {

sendMessage(

id:ID!,

text:String!

):Message

```
}
```

```
type Subscription {
```

```
  onSendMessage:Message
```

```
  @aws_subscribe(
```

```
    mutations:["sendMessage"]
```

```
)
```

```
}
```

Flow

User A:

Runs mutation:

```
mutation {
```

```
  sendMessage(
```

```
    id:"1",
```

```
    text:"Hello"
```

```
)
```

```
}
```

AppSync publishes event.

User B receives:

```
subscription {
```

```
  onSendMessage{
```

```
    text
```

```
  }
```

```
}
```

Troubleshooting

Problem:

Subscription does not receive messages.

Check:

1. Mutation exists.
 2. Subscription references mutation.
 3. Client is connected.
 4. Authentication allows subscription.
-

Lab 4: AppSync with Lambda Resolver

Objective

Use Lambda for custom business logic.

Use Case

Calculate discount.

Input:

```
mutation{
```

```
  calculateDiscount(
```

```
    customerId:"100"
```

```
  )
```

```
}
```

Lambda:

```
def lambda_handler(event,context):
```

```
    customer=event["arguments"]["customerId"]
```

```
    discount=10
```

```
    return {
```

```
        "customerId":customer,
```

```
        "discount":discount
```

```
    }
```

Architecture

Client

|

AppSync

|

Lambda Resolver

|

Business Logic

Lab 5: Enable Monitoring

Objective

Enable production troubleshooting.

Enable:

AppSync

→ Settings

→ Logging

→ CloudWatch Logs

Select:

Development:

ALL

Production:

ERROR

Enable X-Ray:

Settings

→ X-Ray tracing

→ Enable

Expected Monitoring Data

CloudWatch:

Request count

Latency

Errors

X-Ray:

Client

|

AppSync

|

Lambda

|

DynamoDB

13. Production-Level Project

Project: Real-Time E-Commerce Platform Using AWS AppSync

Business Requirement

Build an online shopping platform supporting:

- Product browsing
 - Customer accounts
 - Orders
 - Real-time order updates
 - Notifications
-

Functional Requirements

Customer Features

Users can:

- Register
 - View products
 - Place orders
 - Track order status
-

Admin Features

Admins can:

- Add products
 - Update inventory
 - Monitor orders
-

Architecture

AWS Services Used

Service	Purpose
AppSync	GraphQL API
Cognito	Authentication
DynamoDB	Product/order data
Aurora	Transaction data
Lambda	Business logic
S3	Product images
CloudFront	Global delivery
EventBridge	Events
SNS	Notifications
CloudWatch	Monitoring
CloudTrail	Auditing
KMS	Encryption

Deployment Process

Step 1

Create infrastructure:

Using:

- CDK
 - CloudFormation
-

Step 2

Deploy:

cdk deploy

Step 3

Create users:

Cognito:

Admin

Customer

Step 4

Deploy schema:

schema.graphql

Example Schema

```
type Product {
```

```
  id:ID!
```

```
  name:String!
```

```
  price:Float!
```

```
  inventory:Int!
```

```
}
```

```
type Query{
```

```
products:[Product]
```

```
}
```

```
type Mutation{
```

```
  placeOrder(
```

```
    productId:ID!
```

```
  ):Order
```

```
}
```

```
type Subscription{
```

```
  orderUpdated:Order
```

```
}
```

Security Design

Authentication

Use:

Cognito User Pools

Authorization

Customer:

Read own orders

Admin:

Manage products

Encryption

Use:

KMS customer managed key

Monitoring Design

CloudWatch alarms:

High Error Rate

Condition:

5XXError > threshold

High Latency

Condition:

Latency > acceptable limit

Logging

Enable:

- AppSync logs
 - Lambda logs
 - CloudTrail
-

Cost Estimation Factors

Main cost drivers:

Component	Cost Driver
-----------	-------------

AppSync	API requests
---------	--------------

Subscriptions	Connection minutes
---------------	--------------------

DynamoDB	Reads/writes
----------	--------------

Lambda	Invocations
--------	-------------

CloudFront	Data transfer
------------	---------------

Aurora	Instance usage
--------	----------------

Cost Optimization

Use:

- DynamoDB on-demand for unpredictable traffic
- AppSync caching
- CloudFront caching
- Lambda right sizing

Production Best Practices

1. Use Cognito instead of API keys.
2. Use IAM roles.
3. Enable monitoring.
4. Use schema version control.
5. Automate deployments.
6. Use least privilege permissions.
7. Add query complexity limits.

8. Use pagination.
9. Avoid expensive nested queries.
10. Encrypt sensitive data.

14. AWS AppSync Best Practices

This section covers AWS-recommended approaches and common production patterns.

14.1 Design GraphQL Schemas Carefully

The schema is the contract between:

- Frontend applications
- Backend systems

A poor schema creates:

- Slow queries
- Difficult maintenance
- Breaking changes

Good Practice

Use meaningful object models.

Example:

```
type Customer {
```

```
  id:ID!
```

```
  name:String!
```

```
  orders:[Order]
```

```
}
```

Avoid exposing database structures directly.

Bad:

```
type DynamoDBItem {
```

```
    pk:String
```

```
    sk:String
```

```
    attribute1:String
```

```
}
```

Why?

Because clients should not depend on database implementation.

14.2 Use Pagination for Large Data Sets

Avoid:

```
query {
```

```
    getAllProducts{
```

```
        id
```

```
        name
```

```
}
```

```
}
```

Problem:

- Large response
 - High latency
 - Increased cost
-

Use:

```
query {
```

```
  listProducts(  
    limit:20,  
    nextToken:"token"  
  )  
  
}
```

Benefits:

- Better performance
 - Lower memory usage
 - Better mobile experience
-

14.3 Avoid N+1 Query Problems

Problem:

Example:

Request:

Get 100 customers

Resolver:

Get customer

Get orders

Get customer

Get orders

...

Creates:

201 database calls

Solutions:

- Batch operations
 - Pipeline resolvers
 - Better data modeling
-

14.4 Use Resolver Caching

Useful for:

- Product catalogs
- Public information
- Read-heavy applications

Example:

Without cache:

1000 requests

1000 DynamoDB reads

With cache:

1000 requests

100 DynamoDB reads

14.5 Use Appropriate Authorization

Do not use API keys for:

- Banking applications
- Healthcare
- Enterprise systems

Use:

Scenario	Recommended
Public API	API Key
Mobile users	Cognito
Backend services	IAM
Custom rules	Lambda Authorizer

14.6 Use Infrastructure as Code

Avoid:

Developer manually creates API

Problems:

- Configuration drift
- Difficult rollback
- No version history

Use:

- CloudFormation
 - CDK
 - SAM
-

14.7 Monitor Production APIs

Enable:

- CloudWatch Metrics
- CloudWatch Logs
- CloudTrail
- X-Ray

Monitor:

- Latency
 - Errors
 - Resolver failures
-

14.8 Secure Secrets

Never store:

password="secret123"

Use:

- Secrets Manager
 - Parameter Store
-

14.9 Use IAM Least Privilege

Bad:

```
{  
  "Action": "*",  
  "Resource": "*"   
}
```

Good:

```
{  
  "Action": [  
    "dynamodb:GetItem"  
  ],  
  "Resource":  
    "specific-table"  
}
```

14.10 Version GraphQL Schemas

Use:

- Git
- CI/CD pipelines
- Review processes

Avoid direct production edits.

15. Common AWS AppSync Mistakes

Mistake 1: Using API Keys in Production

Why it happens

API keys are easy to configure.

Problem

Anyone with the key can call the API.

Solution

Use:

- Cognito
- IAM
- OIDC

Mistake 2: Giving AppSync Too Many IAM Permissions

Example:

```
{  
  "Action": "*" }  
}
```

Problem

Security risk.

Solution

Use only:

dynamodb:GetItem

dynamodb:PutItem

Mistake 3: No Resolver Attached

Symptom:

Error:

Cannot return null

Cause:

Schema exists but backend mapping missing.

Solution:

Create resolver.

Mistake 4: Incorrect DynamoDB Key Mapping

Example:

DynamoDB:

customerId

Resolver expects:

id

Result:

No data returned.

Mistake 5: Poor GraphQL Schema Design

Symptoms:

- Too many fields
- Slow queries
- Difficult changes

Solution:

Design APIs around application needs.

Mistake 6: Ignoring Query Complexity

Problem:

A user sends:

customer{

orders{

```
products{
  reviews{
    users{
      }
    }
  }
}
```

Could overload backend.

Solution:

Implement:

- Query limits
- Depth limits

Mistake 7: Not Using Pagination

Problem:

Large responses.

Solution:

Use:

limit

nextToken

Mistake 8: Excessive Lambda Usage

Problem:

Every request triggers Lambda.

Result:

- Higher latency
- Higher cost

Solution:

Use DynamoDB resolvers where possible.

Mistake 9: Poor DynamoDB Data Modeling

Problem:

Using Scan operations frequently.

Solution:

Design partition keys around access patterns.

Mistake 10: Ignoring Subscription Costs

Problem:

Thousands of WebSocket connections.

Solution:

Monitor:

- Connected clients
 - Message volume
-

Mistake 11: No Monitoring

Problem:

Production failures are difficult to diagnose.

Solution:

Enable:

- CloudWatch Logs
- X-Ray

Mistake 12: Hardcoding Credentials

Bad:

```
secret="abc123"
```

Solution:

IAM roles.

Mistake 13: No Encryption Strategy

Problem:

Compliance failures.

Solution:

Use:

- TLS
 - KMS
-

Mistake 14: Changing Production Schema Directly

Problem:

Breaking clients.

Solution:

Use:

- Version control
 - Deployment pipelines
-

Mistake 15: Ignoring Error Handling

Problem:

Poor user experience.

Solution:

Handle:

- Validation errors
 - Authorization failures
 - Backend failures
-

Mistake 16: Using Scan Instead of Query

Problem:

Expensive DynamoDB operations.

Solution:

Use indexed access patterns.

Mistake 17: Not Testing Authorization Rules

Problem:

Users may access unauthorized data.

Solution:

Test:

- Admin
 - Customer
 - Guest
-

Mistake 18: No Backup Strategy

Problem:

Data loss risk.

Solution:

Use:

- DynamoDB backups
 - Point-in-time recovery
-

Mistake 19: Overusing Pipeline Resolvers

Problem:

Complexity.

Solution:

Use simple resolvers where possible.

Mistake 20: No Cost Monitoring

Problem:

Unexpected AWS bills.

Solution:

Use:

- Cost Explorer
 - Budgets
 - Alerts
-

16. AWS Certified Developer Associate (DVA-C02) Exam Tips

16.1 Frequently Tested AppSync Concepts

The exam commonly tests:

GraphQL Operations

Remember:

Operation	Purpose
-----------	---------

Query	Read
-------	------

Mutation	Write
----------	-------

Subscription	Real-time
--------------	-----------

Mnemonic:

QMS

Q = Query

M = Modify

S = Stream

16.2 Resolver Concepts

Remember:

GraphQL Field

|

|

Resolver

|

|

Data Source

Resolver connects API to backend.

16.3 Authentication Questions

Scenario:

"Mobile application with millions of users."

Best answer:

Cognito User Pool

Scenario:

"Lambda needs to call AppSync."

Best answer:

IAM Authentication

Scenario:

"Temporary testing."

Best answer:

API Key

16.4 DynamoDB Integration Questions

Remember:

AppSync + DynamoDB is preferred for:

- Serverless applications
 - CRUD operations
 - High scalability
-

16.5 Real-Time Questions

Keyword:

"Users receive updates instantly"

Answer:

GraphQL Subscription

16.6 Monitoring Questions

Requirement Service

Logs	CloudWatch Logs
Metrics	CloudWatch Metrics
API changes	CloudTrail
Tracing	X-Ray

16.7 Security Exam Traps

Question:

"Developer stores database password in Lambda code."

Correct:

Secrets Manager

Not:

Environment variable with plaintext secret

16.8 Cost Optimization Questions

Scenario:

"Many repeated read requests."

Consider:

AppSync caching

17. AppSync Comparison Tables

AppSync vs API Gateway

Feature	AppSync	API Gateway
API Style	GraphQL	REST/WebSocket

Feature	AppSync	API Gateway
Client controls fields	Yes	No
Real-time	Built-in subscriptions	Requires setup
Multiple data sources	Excellent	Good
Best for	Data-driven apps	REST APIs

AppSync vs Lambda Function URL

Feature	AppSync	Lambda URL
GraphQL	Yes	No
Authentication	Multiple options	IAM only/custom
Data aggregation	Yes	Manual
Real-time	Yes	Limited

AppSync vs API Gateway WebSocket

Feature	AppSync	API Gateway WebSocket
Connection management	Managed	Developer manages logic
GraphQL	Yes	No
Subscriptions	Built-in	Custom
Best for	Real-time apps	Custom messaging

AppSync vs REST API

Feature	REST	AppSync
Data fetching	Fixed response	Client-defined

Feature	REST	AppSync
Endpoint count	Multiple	Usually one
Learning curve	Lower	Higher
Real-time	Additional services	Built-in

18. AWS AppSync Cheat Sheet

Core Concepts

AppSync = Managed GraphQL API Service

Operations

Query → Read

Mutation → Write

Subscription → Real-time

Components

Schema

|

Resolver

|

Data Source

Authentication

API Key

IAM

Cognito

OIDC

Lambda Authorizer

Common Data Sources

DynamoDB

Lambda

Aurora

HTTP Endpoint

OpenSearch

Monitoring

CloudWatch Logs

CloudWatch Metrics

CloudTrail

X-Ray

Security

Remember:

TLS = Encryption in Transit

KMS = Encryption at Rest

IAM = Permissions

Secrets Manager = Credentials

Performance

Remember:

Pagination

Caching

Avoid N+1

Optimize Schema

Exam Memory

Instant updates → Subscription

Mobile users → Cognito

AWS service calls → IAM

Custom logic → Lambda Resolver

CRUD serverless → DynamoDB

19. AWS Certified Developer – Associate (DVA-C02) Scenario-Based Exam Questions: AWS AppSync

These questions are written in the style of the real AWS Certified Developer Associate exam.

Focus areas:

- Architecture decisions
- Security
- Troubleshooting
- Performance
- Cost optimization

- AWS best practices
-

Question 1: Choosing Authentication for a Mobile Application

Scenario

A company is building a mobile banking application using AWS AppSync. Millions of customers will authenticate using usernames and passwords. Each customer should only access their own account information.

Which solution provides the MOST appropriate authentication mechanism?

Options

- A. Use an API key for AppSync authentication.
 - B. Use Amazon Cognito User Pools with AppSync authorization.
 - C. Use IAM authentication for every mobile user.
 - D. Use a shared Lambda authorization token.
-

Correct Answer

- ☒ B. Use Amazon Cognito User Pools with AppSync authorization.
-

Explanation

Cognito provides:

- User authentication
- JWT tokens
- User identity
- User groups
- Integration with AppSync

The application can identify:

Customer A

Customer B

and authorize access.

Why Other Options Are Incorrect

A. API Key

Incorrect.

API keys are:

- Shared
 - Not user-specific
 - Not suitable for banking applications
-

C. IAM Authentication

Incorrect.

IAM is better for:

- AWS services
- Backend applications

Not millions of end users.

D. Shared Lambda Token

Incorrect.

A shared token does not provide proper user identity management.

Exam Concept Tested

AppSync Authentication Modes

Question 2: Real-Time Notifications

Scenario

An application displays live stock price changes to thousands of connected users. Whenever prices change, all connected clients must immediately receive updates.

Which AppSync feature should be used?

Options

- A. GraphQL Query
 - B. GraphQL Mutation
 - C. GraphQL Subscription
 - D. DynamoDB Scan
-

Correct Answer

☒ C. GraphQL Subscription

Explanation

Subscriptions provide:

- Real-time communication
 - WebSocket-based updates
 - Push notifications
-

Incorrect Options

A. Query

Only retrieves data when requested.

B. Mutation

Changes data but does not automatically push updates.

D. DynamoDB Scan

Reads data inefficiently.

Exam Concept Tested

GraphQL Operations

Question 3: Protecting Database Credentials

Scenario

A Lambda resolver connected to AppSync needs database credentials to access Aurora. A developer currently stores the password in environment variables.

What is the BEST solution?

Options

- A. Store the password in Lambda source code.
 - B. Store the password in Amazon S3.
 - C. Store credentials in AWS Secrets Manager.
 - D. Store credentials in a Git repository.
-

Correct Answer

- ☒ C. Store credentials in AWS Secrets Manager.
-

Explanation

Secrets Manager provides:

- Secure storage
- Rotation
- Access control
- Auditing

Incorrect Options

A:

Hardcoded secrets are insecure.

B:

S3 is not designed for secret management.

D:

Source control exposure risk.

Exam Concept Tested

AWS Security Best Practices

Question 4: Reducing DynamoDB Reads

Scenario

A product catalog application uses AppSync with DynamoDB. Thousands of users repeatedly request the same product information.

How should developers reduce DynamoDB costs?

Options

- A. Enable AppSync API caching.
 - B. Increase DynamoDB capacity.
 - C. Replace DynamoDB with RDS.
 - D. Disable resolver caching.
-

Correct Answer

- ☒ A. Enable AppSync API caching.
-

Explanation

Caching reduces:

- DynamoDB requests
 - Latency
 - Cost
-

Incorrect Options

B:

Adds capacity but does not reduce unnecessary reads.

C:

Unnecessary redesign.

D:

Opposite of the requirement.

Exam Concept Tested

Performance Optimization

Question 5: Lambda Resolver Permissions

Scenario

An AppSync API invokes Lambda resolvers. Requests fail with:

AccessDeniedException

What should the developer check?

Options

- A. Lambda memory size
- B. Lambda execution timeout
- C. IAM permissions allowing AppSync to invoke Lambda

D. DynamoDB partition key

Correct Answer

☒ C. IAM permissions allowing AppSync to invoke Lambda

Explanation

AppSync requires:

```
{  
  "Action":  
    "lambda:InvokeFunction"  
}
```

Incorrect Options

A:

Memory affects performance, not permission.

B:

Timeout creates different errors.

D:

Unrelated.

Exam Concept Tested

IAM Permissions

Question 6: API Automation

Scenario

A company needs to create identical AppSync APIs across development, testing, and production environments.

Which approach should be used?

Options

- A. Manually configure APIs in the console.
 - B. Use CloudFormation or CDK.
 - C. Create APIs using screenshots.
 - D. Store settings in documentation only.
-

Correct Answer

- ☒ **B. Use CloudFormation or CDK.**
-

Explanation

Infrastructure as Code provides:

- Repeatability
 - Version control
 - Automation
-

Incorrect Options

A:

Causes configuration drift.

C:

Not automation.

D:

Manual process.

Exam Concept Tested

Infrastructure as Code

Question 7: Slow GraphQL Queries

Scenario

An AppSync API has slow responses. Investigation shows one query triggers hundreds of DynamoDB requests.

What problem is occurring?

Options

- A. API key expiration
 - B. N+1 query problem
 - C. Incorrect IAM policy
 - D. Missing CloudTrail
-

Correct Answer

☒ B. N+1 query problem

Explanation

One request creates many backend calls.

Solutions:

- Batch operations
 - Better schema design
 - Pipeline resolvers
-

Incorrect Options

A:

Authentication issue.

C:

Permission issue.

D:

Auditing service.

Exam Concept Tested

Application Performance Optimization

Question 8: Backend Service Authentication

Scenario

A Lambda function needs to call an AppSync GraphQL API securely.

Which authentication method should be used?

Options

- A. API Key
 - B. IAM Authentication
 - C. Anonymous access
 - D. Hardcoded token
-

Correct Answer

 **B. IAM Authentication**

Explanation

AWS services should authenticate using IAM roles.

Example:

Lambda Role

|

|

appsync:GraphQL

Incorrect Options

A:

Shared secret.

C:

No security.

D:

Bad practice.

Exam Concept Tested

Service-to-Service Authentication

Question 9: File Upload Architecture

Scenario

A mobile application allows users to upload profile images. Developers want to avoid sending large files through AppSync.

What architecture should be used?

Options

- A. Store images directly in AppSync.
 - B. Upload files directly to S3 using pre-signed URLs.
 - C. Store images inside DynamoDB.
 - D. Send files through Lambda.
-

Correct Answer

- ☒ **B. Upload files directly to S3 using pre-signed URLs.**
-

Explanation

Benefits:

- Scalable storage
 - Lower AppSync load
 - Lower cost
-

Incorrect Options

A:

AppSync is not file storage.

C:

DynamoDB is not suitable for large objects.

D:

Lambda has payload limitations.

Exam Concept Tested

Serverless Architecture Design

Question 10: Auditing AppSync Changes

Scenario

A security team wants to know who changed AppSync authentication settings.

Which AWS service should be used?

Options

A. CloudWatch

B. CloudTrail

C. X-Ray

D. Route 53

Correct Answer

 **B. CloudTrail**

Explanation

CloudTrail records:

- API calls
 - Configuration changes
 - User actions
-

Incorrect Options

A:

Monitoring logs.

C:

Performance tracing.

D:

DNS service.

Exam Concept Tested

AWS Auditing

Question 11: Choosing Data Source

Scenario

A company needs a serverless GraphQL API for simple CRUD operations on user profiles.

Which data source is the best fit?

- A. DynamoDB
 - B. EC2 database
 - C. File system
 - D. Manual servers
-

Correct Answer

- ☒ A. DynamoDB
-

Explanation

DynamoDB provides:

- Serverless scaling
 - Low latency
 - Native AppSync integration
-

Question 12: Debugging Resolver Failures

Scenario

A resolver returns incorrect data. Developers need detailed resolver execution information.

What should they enable?

- A. CloudWatch Logs
 - B. Route 53 Resolver
 - C. S3 Versioning
 - D. IAM Access Analyzer only
-

Correct Answer

☒ A. CloudWatch Logs

Question 13: Global User Performance

Scenario

Users worldwide experience latency accessing an AppSync API.

Which service can improve global access performance?

- A. CloudFront
 - B. DynamoDB Streams
 - C. IAM
 - D. Secrets Manager
-

Correct Answer

☒ A. CloudFront

Question 14: Asynchronous Processing

Scenario

An AppSync mutation starts a long-running video processing job.

The API should return immediately.

Which service should handle background processing?

- A. SQS
 - B. API Key
 - C. Cognito
 - D. CloudTrail
-

Correct Answer

☒ A. SQS

Question 15: Event-Driven Architecture

Scenario

Multiple systems need to react when an order is created.

Which service is best for event distribution?

- A. SNS/EventBridge
- B. DynamoDB Scan
- C. AppSync Schema
- D. IAM Policy

Correct Answer

- ☒ A. SNS/EventBridge

Question 16: Encryption Requirements

Scenario

A healthcare application requires customer-managed encryption keys.

Which service should be used?

- A. KMS
- B. CloudFront
- C. SNS
- D. Route 53

Correct Answer

- ☒ A. KMS

Question 17: Monitoring API Latency

Scenario

Developers need to identify slow backend calls.

Which service provides distributed tracing?

- A. X-Ray
 - B. IAM
 - C. S3
 - D. Cognito
-

Correct Answer

- ☒ A. X-Ray
-

Question 18: Protecting Against Excessive Queries

Scenario

Users can submit expensive nested GraphQL queries that increase backend costs.

What should developers implement?

- A. Query complexity limits
 - B. More API keys
 - C. Larger S3 buckets
 - D. More IAM users
-

Correct Answer

- ☒ A. Query complexity limits
-

Question 19: Deployment Consistency

Scenario

A team wants automated deployment of AppSync schema changes.

What should they use?

- A. CI/CD with CloudFormation/CDK
 - B. Manual console edits
 - C. Email approvals only
 - D. Local configuration files only
-

Correct Answer

- ☒ A. CI/CD with CloudFormation/CDK
-

Question 20: Reducing Resolver Complexity

Scenario

A resolver performs simple DynamoDB reads but uses Lambda unnecessarily.

What is the recommended improvement?

- A. Use direct DynamoDB resolver.
 - B. Add more Lambda functions.
 - C. Disable monitoring.
 - D. Use API keys.
-

Correct Answer

- ☒ A. Use direct DynamoDB resolver.
-

20. AWS AppSync Practice Questions

This section contains:

- 20 Multiple Choice Questions
- 10 True/False Questions
- 10 Debugging Questions

- 10 Hands-on Questions
- 10 IAM Questions

Each answer includes an explanation.

Part 1: Multiple Choice Questions (20)

MCQ 1

A developer needs an API where clients can request only the fields they require.

Which technology does AWS AppSync provide?

- A. REST API
- B. GraphQL API
- C. FTP API
- D. SOAP API

Answer

☒ B. GraphQL API

Explanation

GraphQL allows clients to specify required fields.

Example:

```
query {  
  user {  
    name  
  }  
}
```

Only:

```
{  
  "name": "Alice"  
}
```

is returned.

MCQ 2

Which AppSync operation is used to retrieve data?

- A. Mutation
- B. Subscription
- C. Query
- D. Resolver

Answer

☒ C. Query

Explanation:

Operation	Purpose
-----------	---------

Query	Read
-------	------

Mutation	Write
----------	-------

Subscription	Real-time
--------------	-----------

MCQ 3

Which AppSync feature enables real-time updates?

- A. Query
- B. Subscription
- C. Schema
- D. IAM Role

Answer

☒ B. Subscription

Explanation:

Subscriptions maintain WebSocket connections.

MCQ 4

A developer wants AppSync to access DynamoDB.

What connects GraphQL fields to DynamoDB?

- A. IAM User
- B. Resolver
- C. CloudTrail
- D. CloudFront

Answer

☒ B. Resolver

Explanation:

Resolver maps:

GraphQL Field

↓

Backend Data Source

MCQ 5

Which service is commonly used for AppSync user authentication?

- A. Cognito User Pools
- B. S3
- C. ECR
- D. Route 53

Answer

☒ A. Cognito User Pools

MCQ 6

Which AWS service records AppSync configuration changes?

- A. CloudWatch
- B. CloudTrail
- C. X-Ray
- D. SNS

Answer

- ☒ B. CloudTrail
-

MCQ 7

A Lambda resolver cannot call another AWS service.

What should be checked first?

- A. IAM role permissions
- B. DNS settings
- C. S3 bucket name
- D. CloudFront distribution

Answer

- ☒ A. IAM role permissions
-

MCQ 8

Which service stores application secrets securely?

- A. DynamoDB
- B. Secrets Manager
- C. CloudFormation
- D. API Gateway

Answer

- ☒ B. Secrets Manager
-

MCQ 9

A company needs infrastructure version control for AppSync.

Which solution should be used?

- A. Manual console configuration
- B. CloudFormation/CDK
- C. Spreadsheet tracking
- D. Email documentation

Answer

- ☒ B. CloudFormation/CDK
-

MCQ 10

Which authentication method is best for Lambda calling AppSync?

- A. API Key
- B. IAM Authentication
- C. Anonymous access
- D. Password sharing

Answer

- ☒ B. IAM Authentication
-

MCQ 11

A product catalog has millions of repeated reads.

What reduces backend requests?

- A. AppSync caching
- B. Disable DynamoDB
- C. Remove resolvers
- D. Increase API keys

Answer

- ☒ A. AppSync caching
-

MCQ 12

Which service provides distributed tracing?

- A. CloudWatch
- B. X-Ray
- C. IAM
- D. KMS

Answer

☒ B. X-Ray

MCQ 13

Which service is best for storing images uploaded by users?

- A. DynamoDB
- B. S3
- C. AppSync
- D. CloudTrail

Answer

☒ B. S3

MCQ 14

Which AppSync component defines available API operations?

- A. Schema
- B. Resolver
- C. IAM Policy
- D. Lambda Layer

Answer

☒ A. Schema

MCQ 15

Which service provides event routing for AppSync applications?

- A. EventBridge
- B. KMS

- C. IAM
- D. Route 53

Answer

- ☒ A. EventBridge
-

MCQ 16

A developer wants to execute custom business logic after a mutation.

What should be used?

- A. Lambda resolver
- B. API Key
- C. S3 bucket
- D. CloudFront

Answer

- ☒ A. Lambda resolver
-

MCQ 17

Which encryption service manages customer-controlled keys?

- A. KMS
- B. SNS
- C. SQS
- D. ECS

Answer

- ☒ A. KMS
-

MCQ 18

A query triggers hundreds of backend requests.

What problem exists?

- A. API key problem
- B. N+1 query problem
- C. DNS problem
- D. IAM problem

Answer

- ☒ B. N+1 query problem
-

MCQ 19

Which service provides queue-based asynchronous processing?

- A. SQS
- B. Cognito
- C. CloudTrail
- D. KMS

Answer

- ☒ A. SQS
-

MCQ 20

Which AppSync data source is best for simple serverless CRUD?

- A. DynamoDB
- B. EC2 database server
- C. Local file storage
- D. Manual database

Answer

- ☒ A. DynamoDB
-

Part 2: True / False Questions (10)

1

AWS AppSync supports GraphQL subscriptions.

Answer:

☒ True

Explanation:

Subscriptions provide real-time updates.

2

API keys are recommended for highly sensitive banking applications.

Answer:

☒ False

Explanation:

Use Cognito or IAM.

3

Resolvers connect GraphQL fields with backend services.

Answer:

☒ True

4

CloudTrail provides distributed request tracing.

Answer:

☒ False

Explanation:

X-Ray provides tracing.

5

DynamoDB is a common AppSync data source.

Answer:

☒ True

6

AppSync requires developers to manage WebSocket servers.

Answer:

☒ False

Explanation:

AWS manages subscription infrastructure.

7

IAM authentication is suitable for AWS service-to-service communication.

Answer:

☒ True

8

Secrets Manager is used for storing database passwords.

Answer:

☒ True

9

GraphQL queries modify database records.

Answer:

☒ False

Explanation:

Mutations modify data.

10

CloudFormation can deploy AppSync infrastructure.

Answer:

☒ True

Part 3: Debugging Questions (10)

Debugging 1

Problem:

GraphQL query returns:

Cannot return null for non-nullable field User.email

What should you check?

Answer:

Check:

- DynamoDB item contains email
- Resolver mapping
- Schema requirement

Cause:

Schema:

email:String!

requires a value.

Debugging 2

Problem:

AppSync returns:

AccessDeniedException

when reading DynamoDB.

Solution:

Check AppSync IAM role.

Required:

"dynamodb:GetItem"

Debugging 3

Problem:

Subscription receives no messages.

Check:

- Mutation exists
 - Subscription mapping exists
 - Client connection
 - Authorization
-

Debugging 4

Problem:

API is slow.

Investigate:

- Resolver latency
 - Lambda duration
 - DynamoDB performance
 - X-Ray traces
-

Debugging 5

Problem:

Users receive unauthorized errors.

Check:

- JWT token
 - Cognito configuration
 - Authorization rules
-

Debugging 6

Problem:

Schema deployment fails.

Check:

- Syntax errors
 - Missing Query root
 - Invalid types
-

Debugging 7

Problem:

Lambda resolver timeout.

Check:

- Lambda timeout
 - External API latency
 - Memory allocation
-

Debugging 8

Problem:

Unexpected AWS costs.

Check:

- Subscription connections
 - Request volume
 - Cache usage
 - Data transfer
-

Debugging 9

Problem:

DynamoDB returns no items.

Check:

- Partition key
 - Resolver mapping template
 - Item format
-

Debugging 10

Problem:

Production API changes unexpectedly.

Check:

- Manual console changes
 - CI/CD process
 - CloudFormation drift
-

Part 4: Hands-on Questions (10)

Create an AppSync API connected to DynamoDB.

Expected steps:

1. Create DynamoDB table.
 2. Create GraphQL schema.
 3. Create data source.
 4. Attach resolver.
 5. Test query.
-

2

Add authentication.

Expected:

Configure Cognito User Pool authorization.

3

Create real-time updates.

Expected:

Create:

- Mutation
 - Subscription
-

4

Add Lambda resolver.

Expected:

1. Create Lambda.
2. Add AppSync permission.
3. Attach resolver.

5

Enable monitoring.

Expected:

Enable:

- CloudWatch Logs
- Metrics
- X-Ray

6

Deploy using CDK.

Expected:

Use:

cdk deploy

7

Deploy using CloudFormation.

Expected:

Create stack:

aws cloudformation deploy

8

Secure database credentials.

Expected:

Use:

Secrets Manager

9

Improve performance.

Expected:

Use:

- Pagination
- Cache
- Better schema

10

Create CI/CD.

Expected:

Use:

- CodePipeline
- CloudFormation/CDK

Part 5: IAM Questions (10)

IAM Question 1

What permission allows GraphQL execution?

Answer:

appsync:GraphQL

IAM Question 2

What permission allows AppSync to invoke Lambda?

Answer:

lambda:InvokeFunction

IAM Question 3

What principle should IAM follow?

Answer:

Least privilege.

IAM Question 4

Should Lambda credentials be hardcoded?

Answer:

No.

Use IAM roles.

IAM Question 5

What IAM component gives AWS services permissions?

Answer:

IAM Role.

IAM Question 6

What permission allows DynamoDB reads?

Example:

dynamodb:GetItem

IAM Question 7

What authentication should backend AWS services use?

Answer:

IAM.

IAM Question 8

What stores application secrets?

Answer:

Secrets Manager.

IAM Question 9

What records IAM/API changes?

Answer:

CloudTrail.

IAM Question 10

What prevents excessive permissions?

Answer:

Least privilege policies.

21. AWS AppSync Interview Questions (Beginner to Advanced)

This section prepares you for AWS Developer interviews and cloud engineering discussions.

Beginner Level Questions

1. What is AWS AppSync?

Answer

AWS AppSync is a managed service that makes it easy to build GraphQL APIs.

It provides:

- GraphQL API management
- Real-time subscriptions
- Data source integration
- Authentication
- Authorization
- Caching

Developers do not need to manage:

- Servers
 - API infrastructure
 - WebSocket infrastructure
-

2. Why was AWS AppSync created?

Answer

Traditional APIs often require:

- Multiple endpoints
- Multiple backend calls
- Custom real-time solutions

Example:

REST application:

GET /users

GET /orders

GET /payments

The frontend must call multiple APIs.

AppSync allows:

```
query {  
  user {  
    orders {  
      paymentStatus  
    }  
  }  
}
```

A single request can retrieve related data.

3. What is GraphQL?

Answer

GraphQL is an API query language where clients specify exactly what data they need.

Example:

Request:

```
query {  
  product {  
    name  
    price  
  }  
}
```

Response:

```
{  
  "name": "Laptop",  
  "price": 1000  
}
```

Advantages:

- Avoids over-fetching
 - Avoids under-fetching
 - Single API endpoint
-

4. What are the three GraphQL operations?

Answer

Operation	Purpose
-----------	---------

Query	Read data
-------	-----------

Mutation	Create/update/delete data
----------	---------------------------

Subscription	Real-time updates
--------------	-------------------

Memory trick:

Q = Query = Question

M = Mutation = Modify

S = Subscription = Stream

5. What is an AppSync Resolver?

Answer

A resolver connects a GraphQL operation to a backend data source.

Architecture:

GraphQL Field

|

Resolver

|

Data Source

Example:

getUser()

|

DynamoDB GetItem

6. What data sources can AppSync use?

Answer

Common data sources:

- DynamoDB
- Lambda
- Aurora Serverless
- HTTP endpoints
- OpenSearch
- EventBridge integrations

7. What is an AppSync subscription?

Answer

A subscription provides real-time updates using WebSocket connections.

Example:

Chat application:

User A sends message

|

AppSync

|

User B instantly receives message

8. What authentication methods does AppSync support?

Answer

AppSync supports:

Authentication	Use Case
API Key	Testing/public APIs
Cognito User Pool	User applications
IAM	AWS service access
OIDC	External identity providers
Lambda Authorizer	Custom authorization

9. When should you use Lambda with AppSync?

Answer

Use Lambda when:

- Business logic is complex
- External APIs must be called
- Data transformation is required

Example:

AppSync

|

Lambda

|

Payment Service

10. When should you avoid Lambda resolvers?

Answer

Avoid Lambda when:

- Simple DynamoDB operations are enough

Example:

Instead of:

AppSync

|

Lambda

|

DynamoDB

Use:

AppSync

|

DynamoDB Resolver

Benefits:

- Lower latency
- Lower cost

Intermediate Level Questions

11. Explain AppSync request flow.

Answer

Flow:

Client

|

GraphQL Request

|

AppSync

|

Authentication

|

Resolver

|

Data Source

|

Response

Steps:

1. Client sends query.
2. AppSync authenticates request.
3. Resolver executes.
4. Backend returns data.
5. AppSync formats GraphQL response.

12. How does AppSync integrate with DynamoDB?

Answer

AppSync can directly connect to DynamoDB.

Operations:

GraphQL DynamoDB

Query GetItem/Query

Mutation PutItem/UpdateItem/DeleteItem

No Lambda is required.

13. Explain AppSync caching.

Answer

AppSync caching stores frequently requested responses.

Useful for:

- Product catalogs
- Public content
- Read-heavy APIs

Benefits:

- Lower latency
 - Lower DynamoDB costs
 - Reduced backend load
-

14. What is a pipeline resolver?

Answer

A pipeline resolver executes multiple functions sequentially.

Example:

Request

|

Validate User

|

Check Permission

|

Fetch Data

|

Transform Response

Used for:

- Complex workflows
- Multiple backend operations

15. How do you secure AppSync APIs?

Answer

Use:

- Strong authentication
- IAM least privilege
- Cognito authorization
- Encryption
- Monitoring
- Schema controls

16. How do you monitor AppSync?

Answer

Use:

CloudWatch

For:

- Logs
- Metrics
- Errors

CloudTrail

For:

- API changes
- Administrative actions

X-Ray

For:

- Request tracing

17. How would you troubleshoot slow AppSync responses?

Answer

Steps:

1. Check AppSync latency metrics.
 2. Review CloudWatch logs.
 3. Enable X-Ray.
 4. Analyze resolver performance.
 5. Check DynamoDB/Lambda latency.
-

18. Explain N+1 query problem.

Answer

Example:

Request:

Get 100 users

Resolver:

Get user

Get orders

Get user

Get orders

...

Creates hundreds of database calls.

Solutions:

- Batch requests
- Better schema design
- Pipeline resolvers

19. How do you deploy AppSync APIs?

Answer

Use:

- CloudFormation
- AWS CDK
- AWS SAM

Benefits:

- Automation
- Version control
- Repeatability

20. How would you design a highly available AppSync application?

Answer

Architecture:

Users

|

CloudFront

|

AppSync

|

Multiple Data Sources

|

DynamoDB Global Tables

Consider:

- Multi-region data
- Authentication redundancy
- Monitoring
- Automated deployment

Advanced Interview Questions

21. Explain AppSync authorization with Cognito.

Answer

Flow:

User Login

|

Cognito

|

JWT Token

|

AppSync

|

Resolver Authorization

AppSync validates:

- Token signature
 - User identity
 - Claims
-

22. How do you implement field-level authorization?

Answer

Use:

- GraphQL directives
- Resolver logic
- Cognito groups
- Lambda authorization

Example:

Admin fields:

@aws_cognito_user_pools

23. How would you design a multi-tenant AppSync application?

Answer

Approach:

Use tenant isolation.

Example DynamoDB key:

PK:

TENANT#123

SK:

USER#456

Authorization:

User tenant ID

must match

Data tenant ID

24. How do you handle file uploads in AppSync?

Answer

Recommended architecture:

Client

|

AppSync

|

Generate S3 Pre-Signed URL

|

S3 Upload

Do not send large files through AppSync.

25. How do you reduce AppSync costs?

Answer

Methods:

- Enable caching
 - Optimize queries
 - Reduce unnecessary Lambda calls
 - Use efficient DynamoDB keys
 - Monitor usage
-

26. How do you prevent expensive GraphQL queries?

Answer

Implement:

- Query depth limits
 - Complexity analysis
 - Pagination
 - Authorization rules
-

27. Explain AppSync with EventBridge.

Answer

Pattern:

Application Event

|

EventBridge

|

Lambda

|

AppSync

|

Clients

Used for:

- Event-driven applications
- Decoupled systems

28. Explain AppSync with Step Functions.

Answer

Used for workflows.

Example:

Order Mutation

|

Step Functions

|

Payment

|

Inventory

|

Shipping

29. How do you secure sensitive data?

Answer

Use:

- TLS for data in transit
 - KMS encryption
 - Secrets Manager
 - IAM policies
 - Audit logging
-

30. What are common AppSync production mistakes?

Answer

Common mistakes:

1. Using API keys for sensitive applications.

2. Over-permissive IAM policies.
 3. Poor GraphQL schema design.
 4. Missing monitoring.
 5. No pagination.
 6. Excessive Lambda usage.
 7. Hardcoded secrets.
 8. No CI/CD deployment process.
-

22. AWS AppSync Revision Notes

What is AppSync?

Managed AWS GraphQL service.

Remember:

AppSync = GraphQL + Serverless Backend Integration + Real-Time

Core Components

Schema

Defines API structure.

Example:

```
type User {  
  id:ID!  
  name:String  
}
```

Resolver

Connects fields to data.

Schema

|

Resolver

|

Backend

Data Sources

Examples:

- DynamoDB
- Lambda
- Aurora
- HTTP

GraphQL Operations

Operation	Meaning
-----------	---------

Query	Read
-------	------

Mutation	Write
----------	-------

Subscription	Real-time
--------------	-----------

Authentication

Remember:

Public API

|

API Key

Mobile Users

|

Cognito

AWS Services

|

IAM

Custom Logic

|

Lambda Authorizer

Common Architecture

Mobile/Web

|

CloudFront

|

AppSync

|

Resolver

|

DynamoDB/Lambda/Aurora

Security Checklist

- ✓ Use least privilege IAM
 - ✓ Use Cognito for users
 - ✓ Use KMS encryption
 - ✓ Use Secrets Manager
 - ✓ Enable CloudTrail
 - ✓ Enable CloudWatch logs
-

Performance Checklist

- ✓ Pagination
 - ✓ Resolver optimization
 - ✓ Caching
 - ✓ Avoid N+1 queries
 - ✓ Efficient DynamoDB design
-

Troubleshooting Checklist

Authentication Errors

Check:

- JWT token
 - Cognito configuration
 - IAM permissions
-

Resolver Errors

Check:

- Mapping templates
 - Data source permissions
 - Backend availability
-

Slow API

Check:

- Resolver latency
 - Lambda duration
 - DynamoDB performance
 - X-Ray traces
-

Exam Memory Table

Question Keyword	Answer
Real-time updates	Subscription
Mobile authentication	Cognito
AWS service calling AppSync IAM	
Custom logic	Lambda
CRUD API	DynamoDB

Question Keyword	Answer
Audit changes	CloudTrail
Trace requests	X-Ray
Store secrets	Secrets Manager
Encrypt keys	KMS
Infrastructure automation	CDK/CloudFormation

23. Final Knowledge Check

Answer these 10 questions.

I will evaluate your answers, identify weak areas, and recommend what to revise.

Question 1

What are the three GraphQL operations supported by AWS AppSync?

Question 2

A mobile application has millions of users. Which AppSync authentication method should be used and why?

Question 3

Explain the role of an AppSync resolver.

Question 4

A user sends a mutation and other users should instantly receive the update. Which AppSync feature is required?

Question 5

An AppSync Lambda resolver receives `AccessDeniedException` when calling Lambda. What permission is missing?

Question 6

Why should developers avoid using Lambda resolvers for simple DynamoDB CRUD operations?

Question 7

Which AWS service should store database credentials used by Lambda?

Question 8

A GraphQL query creates hundreds of database calls. What problem is this and how can it be fixed?

Question 9

Which AWS services should be used to monitor and audit AppSync applications?

Question 10

Design a simple architecture for a real-time chat application using AppSync.
